

# Bug Report: ML Model CI YAML Workflow

This report details all bugs found in the original YAML workflow file and their solutions.

## Original Issues and Solutions

### Bug 1: Missing Indentation

**Problem:** The YAML file had no indentation, which is required in YAML syntax.

Example from original:

```
name: ML Model CI
on:
push:
branches: main
```

**Solution:** Applied proper 2-space indentation for all nested elements:

```
name: ML Model CI
on:
  push:
    branches-ignore: [main]
  pull_request:
```

### Bug 2: Empty Linter Check Step

**Problem:** The “Linter Check” step had no command to execute.

Original:

```
- name: Linter Check
```

**Solution:** Added flake8 commands with appropriate checks:

```
- name: Linter Check
  run: |
    pip install flake8
    flake8 . --count --select=E9,F63,F7,F82 --show-source --statistics
    flake8 . --count --exit-zero --max-complexity=10 --max-line-length=127 --
statistics
```

### Bug 3: Incorrect Branch Trigger

**Problem:** The workflow was set to run only on the “main” branch, but the task required running on all branches except main.

Original:

```
on:
  push:
    branches: main
```

**Solution:** Changed to branches-ignore to exclude main:

```
on:
  push:
    branches-ignore: [main]
  pull_request:
```

### Bug 4: Missing Artifact Upload Step

**Problem:** The workflow did not include uploading README.md as a project artifact.

**Solution:** Added final step using actions/upload-artifact:

```
- name: Upload Project Documentation
  uses: actions/upload-artifact@v4
  with:
    name: project-doc
    path: README.md
```

### **Bug 5: Incomplete YAML Structure**

**Problem:** The original YAML was missing some required fields and had broken formatting.

**Solution:** Ensured all steps have “uses” or “run” fields, and properly structured the workflow:

- All steps include either “uses” or “run”
- Proper nesting of jobs and steps
- Complete YAML structure with all required fields

### **Summary of Fixes**

- Fixed indentation throughout (2 spaces)
- Added linter commands to check Python code quality
- Modified trigger to run on all branches except main
- Added artifact upload for README.md documentation
- Completed missing YAML structure elements
- Added requirements.txt to ensure dependencies are installed

The final workflow now:

- Triggers on push to any branch except main, plus pull requests
- Sets up Python 3.10 environment
- Installs dependencies from requirements.txt
- Runs comprehensive linting checks
- Verifies PyTorch/model environment
- Uploads README.md as an artifact named “project-doc”